

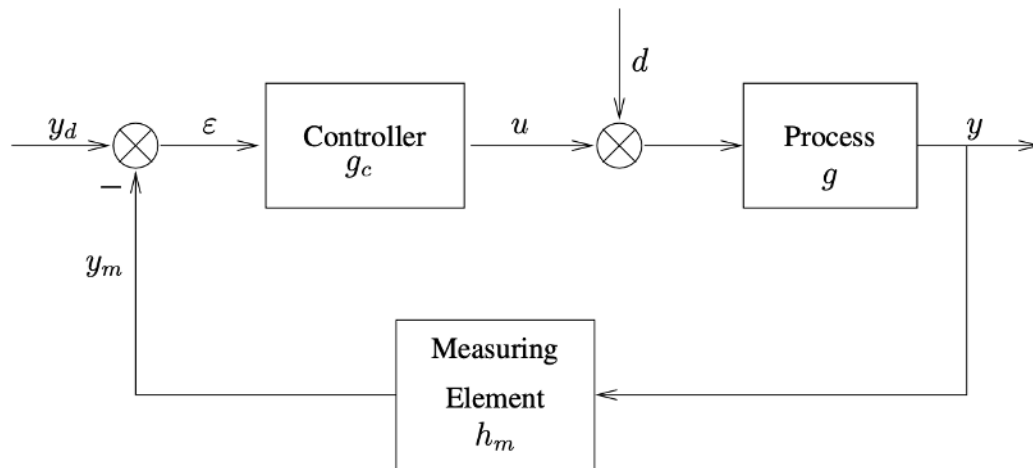
Lab Description

In this lab, students will gain hands-on experience in feedback control using an Arduino Uno and MATLAB Simulink. They will learn how to assemble feedback control systems and investigate tuning parameters for optimal control performance. Throughout the lab, students will interact with hardware setups involving scales, pumps, and breadboards, ensuring correct wiring and configuration. By the end, students will have developed practical skills in feedback control implementation and parameter tuning using real-world hardware and simulation tools.

Learning Outcomes

- Assemble feedback process controls schemes
- Investigate tuning parameters of feedback process control

Theory



Block diagram of a simple feedback control system

y - output variable (temperature, pressure, flow, pH, etc.)

y_d - set point (reference or desired value of controlled variable)

u - output of controller to final control element, e.g. valve (air pressure, current or voltage)

y_m - measured value of output variable

ϵ - error = $y_d - y_m$

d - disturbance to process

A common control law is the Proportional-Integral-Derivative, or PID, controller which has transfer function form:

$$g_c = K_c + \frac{1}{\tau_I s} + \tau_D s \quad [3.1]$$

And when multiplied by the error and put into the time domain:

$$u = K_c \epsilon(t) + \frac{1}{\tau_I} \int_0^t \epsilon(t) dt + \tau_D \frac{d\epsilon}{dt} \quad [3.2]$$

The first term is the proportional term which states the greater the error, the greater the corrective response. The second term is the integral term which adds a corrective term proportional to the integral of the past history of the error and helps ensure that the final output will be exactly the desired value. The final term adds a corrective input proportional to the rate of change of the error, so for processes moving quickly away from the desired value a greater corrective action is taken. The addition of derivative control action can help to stabilize the controller; however, in cases where the measurement is noisy this term can also cause problems.

The problem remains of what the values of the controller gain, K_c , the integral time, τ_I , and the derivative time, τ_D , should be. This is known as controller tuning and is often accomplished by approximating the process transfer function as a first-order system with time delay. The parameters of this approximate transfer function are obtained from the graph of the unit step response, which is known as the process reaction curve, and are used to calculate optimal controllers using various rules of thumb. Mainly, these rules minimize some measure of the total error (eg. sum of square error) of the closed-loop system response to input changes. In this lab, we will utilize PI control, which is a widely adopted method for level control applications.

Steady State Offset with P Control

Steady-state error in a proportional (P) control system refers to the error that remains when the system reaches a stable condition and the output is not exactly equal to the desired setpoint. This error occurs because the proportional control action alone may not be sufficient to completely eliminate the error. Which we derived as follows:

$$\epsilon_{ss} = (y_d - y_{ss}) = (y_d - \frac{K_c y_d}{1+K_c}) \quad [3.3]$$

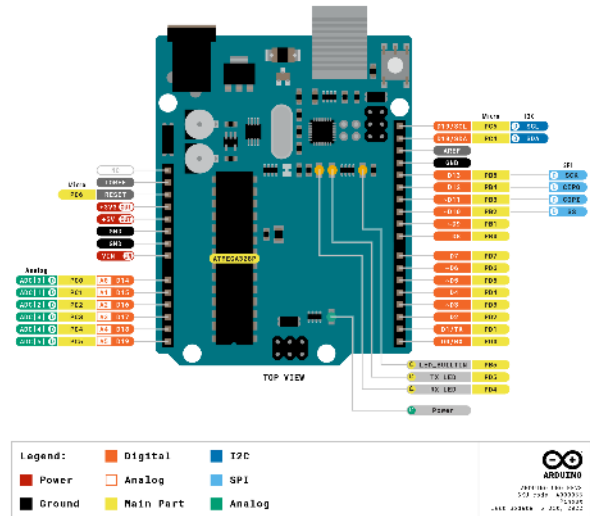
We see that with higher gain K_c the lesser the steady state error.

Materials Required



20kg Digital Load Cell Weight Sensor

The 20kg Digital Load Cell Weight Sensor uses a strain gauge attached to a sturdy aluminum base to measure force or weight. When force is applied, the strain gauge's resistance changes, producing a different electrical output. Note that calibration with known weights is required after installation.

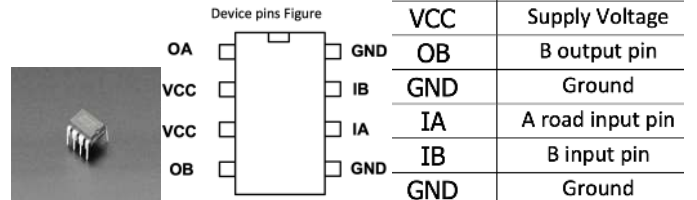


Arduino UNO

The Arduino UNO is a microcontroller board based on the ATmega328P microcontroller. It works by executing instructions stored in its memory, which are written in the Arduino programming language (based on C/C++). The board has digital and analog input/output pins that can be used to interface with various sensors and actuators. These pins can read digital or analog signals from sensors and output signals to control actuators. By writing code and uploading it to the Arduino UNO, users can create interactive electronic projects that respond to their environment.

L9110H H-Bridge Motor Driver x2

The L9110H H-Bridge Motor Driver is a chip that can control the direction and speed of a DC motor. It works by using four transistors to control the flow of current through the motor. By applying different combinations of signals to its input pins, the L9110H can make the motor turn in forward, reverse, or stop it altogether. It is commonly used in robotics and other projects requiring motor control.



NAU7802 24-Bit ADC

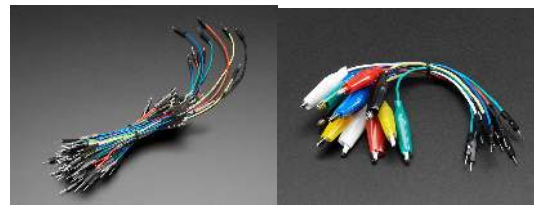
A high-resolution sensor that can measure tiny changes in voltage. It's designed to work with sensors like strain gauges or load cells, which are often used to measure stress or strain. The ADC converts these voltage changes into digital values that can be read by a microcontroller, like an Arduino.





JST SH 4-Pin to Premium Male Headers Cable

A connector cable that allows you to easily connect Qwiic-compatible devices to your projects.



Jumper Wires and Alligator Clips

Used to make temporary connections in breadboards, circuits, or between components.



Half Sized Premium Breadboard with 400 tie points

A prototyping board used to build and test electronic circuits. It consists of interconnected metal clips housed in a plastic case, providing a platform for mounting and connecting electronic components without the need for soldering.



Submersible 3V DC Water Pump
x2

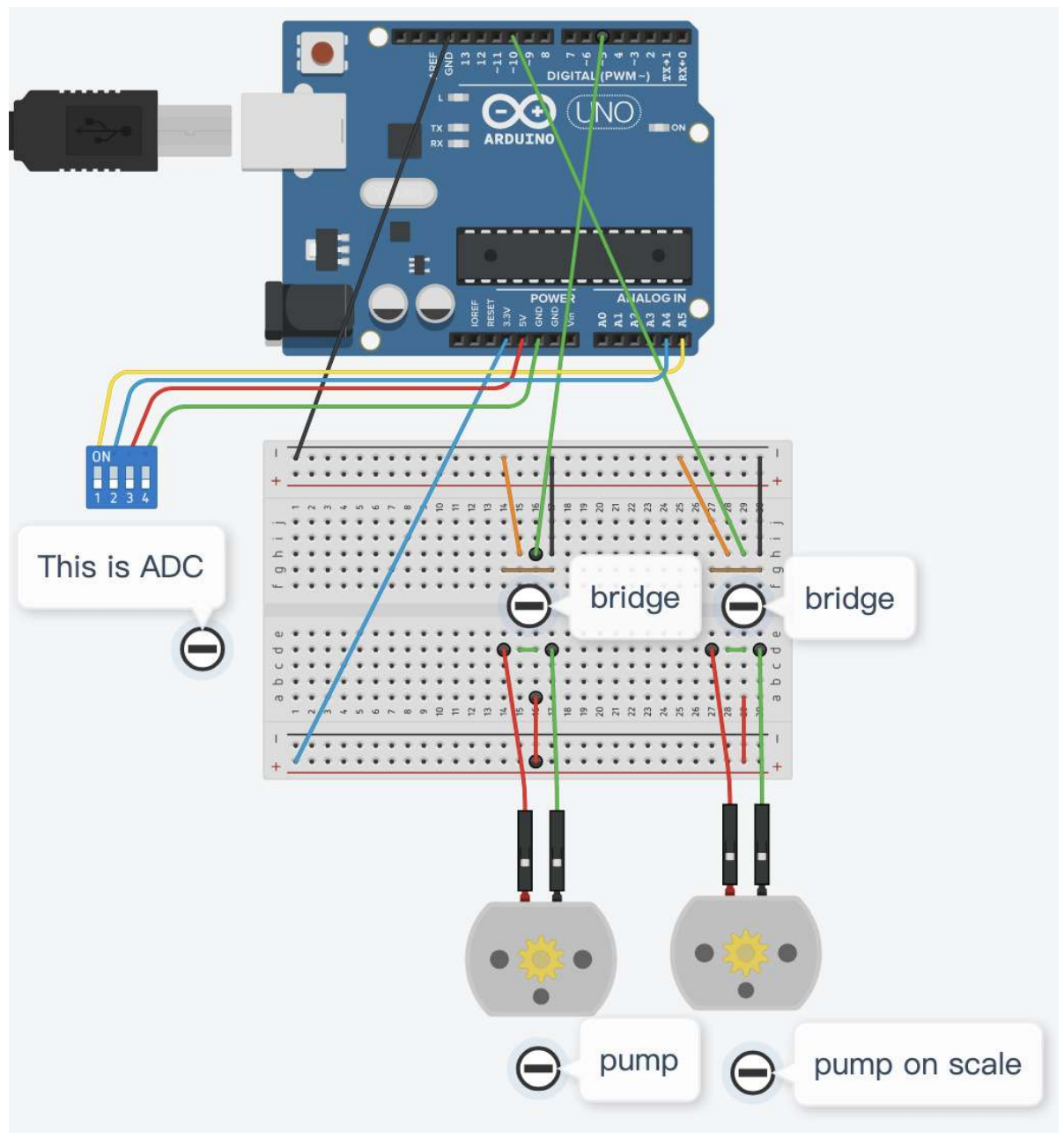
A small water pump designed to be submerged in water and operated using a 3V DC power source. Can pump water to a maximum of 1 meter high.

Other:

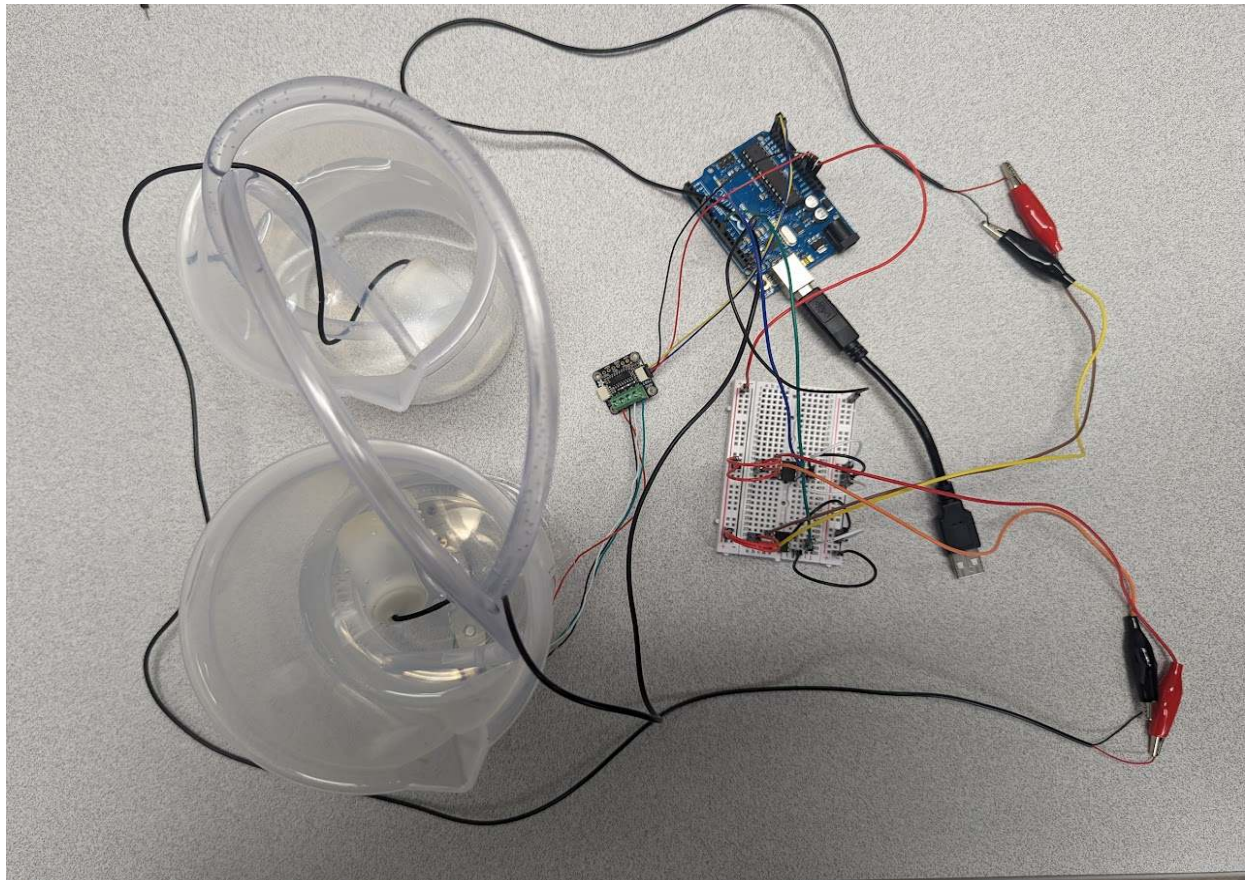
500 mL Beaker (x2), Water, and Tubing for Submersible Pumps (PVC 6mm ID)

Building the Circuit

Use the circuit diagram below to connect the pump, H-Bridge, and scale to the Arduino Uno using the breadboard. **Make sure the notch of the H-Bridge is pointing to the left for the following graph.**



The completed circuit should look something like this:



Code Required

The Arduino Uno and Simulink communicate to one another using serial communication. For this lab, there are two sets of Arduino sketches and Simulink simulations, the procedure for establishing communication and a description of the files is below:

To set up the serial communication, follow the steps below:

1. Download the Arduino sketch and MATLAB Simulink file.
2. Use the Arduino IDE to open the sketch.
3. Connect the Arduino Uno to the computer and select the board (make note of the COM#).
4. Upload the sketch to the Arduino Uno.
5. Open the MATLAB Simulink file

6. Change the COM# on the serial configuration, serial receive, and serial send blocks

Setup

Arduino Sketch (Lab_3_Arduino_Setup.ino):

- Read the value from the scale and write it to Simulink.
- Read the set point power percentage from Simulink to set the inlet pump power.
- Be sure to download the Adafruit NAU7802 Library in the Arduino IDE.

MATLAB Simulink Simulation (Lab_3_Simulink_Setup.slx):

- Receive the scale value from Arduino for visualization and calibration.
- Send the power percentage for the outlet pump to Arduino to further set the pump power.

Controlling the System

Arduino Sketch (Lab_3_Arduino.ino):

- Read the value from the scale and write it to Simulink.
- Set outlet pump power according to some disturbance profile.
- Receive the inlet pump power from Simulink to set the pump power.

MATLAB Simulink Simulation (Lab_3_Simulink.slx):

- Receive the scale value from Arduino for data logging.
- Control scale to be at a given setpoint by manipulating inlet pump power.
- Send the power for the inlet pump to Arduino to further set the pump power.

MATLAB Data Processing and Saving (Lab_3_MATLAB.m):

- Visualize and save system data to a csv and png file for analysis.

Procedure

1. Make the Arduino connection with the scale, pumps, and breadboard.
2. Be sure to have a TA check everything is wired correctly.
3. Connect the Arduino to your computer.
4. Open the "Lab_3_Arduino_Setup.ino" file.

5. Select the correct COM port in the Arduino IDE and upload the code to the Arduino.
6. Open the "Lab_3_Simulink_Setup.slx" file in Simulink and change the COM ports accordingly.
7. Run the file to check if the scale is working properly.
8. If the scale is running properly, put an empty beaker with a pump in it on the scale and record the measurement.
9. Fill the beaker to the 500 mL mark with water and the pump in it and record the measurement.
10. Open the "Lab_3_Arduino.ino" file in Arduino IDE.
11. Change the scale reading (empty and full) with your recorded scale readings in the code.
12. Set up all the equipment as shown in the diagram and fill both beakers to around 300 mL.
13. Before proceeding, have one of the TAs check everything is set up properly.
14. Select the correct COM port.
15. Upload the Arduino code.
16. Open the "Lab_3_Simulink.slx" Simulink file and change the COM ports accordingly.
17. On the left bottom of the Simulink file, you can see the fullness setpoint (%) block. This is the level of the beaker (on the scale) that we need to control.
18. Now on the right bottom of the Simulink diagram, you can see the PI controller. Set $P = 5$ and $I = 0$. Put the 50% value as a setpoint and run the code.
19. The inlet pump will try to control the level. Observe how the inlet pump power varies according to the pump level.
20. If the level of the beaker attains the steady state, then change the setpoint value to 60% or 70%. The controller will act accordingly and try to match the setpoint value.
21. Stop the Simulink simulation when you reach the new steady state (make sure that you will not stop early because it might be that your response will overshoot and after some time it stabilizes).
22. Now run the "Lab_3_MATLAB.m" script. Give your P and I values and get the data and plots. Record the RMSE value and the response time (how much time it takes to reach a new steady state) from the plots.

23. Repeat the procedure from step 17 by changing different P and I values. Start with only the P controller with values $P=1$ and 10 ($I=0$).
24. Repeat these steps for the PI controller with different P and I values e.g., $P=[1,5,10]$ and $I=[0.1,2,5]$.
25. Try different values to try to get the “best” response possible.

Submission

For this submission you will need to include a pdf containing

- A description of the experimental setup.
- A block diagram of the system identifying the controller, sensor, actuator, etc.
- A table of P and I values explored and any summary data chosen.
 - Response time?
 - Root mean square error?
 - Overshoot?
 - Steady state offset?
 - ...
- Plots of the system response for the different tuning parameters.
- Explanation of trends observed for the different P and I values.
- Selection of the “best” P and I values for the system and why they were chosen.